Experiment 1: Build Data Warehouse and Explore WEKA

Objective

- Understand the concept of a data warehouse.
- Build a simple data warehouse from sample data.
- Learn to load and explore data in WEKA.

Tools

- Database tool (e.g., MySQL / PostgreSQL / SQLite) or CSV files
- WEKA (latest version)

Procedure

Part A: Build a Simple Data Warehouse

1. Design a sample schema

Example: `Sales` data warehouse with:

- `Product(product_id, product_name, category)`
- `Customer(customer_id, name, city)`
- Sale(sale_id, product_id, customer_id, date, quantity, price)

2. **Create tables and load sample data**

   Using SQL (example for SQLite):

```
CREATE TABLE Product (
  product_id INTEGER,
  product_name TEXT,
  category TEXT
);

CREATE TABLE Customer (
  customer_id INTEGER,
  name TEXT,
  city TEXT
);

CREATE TABLE Sale (
  sale_id INTEGER,
  product_id INTEGER,
```

```
  customer_id INTEGER,
  date TEXT,
  quantity INTEGER,
  price REAL
);
```

Insert sample rows (5–10 per table).

3. **Create a star-schema view** (for analysis):

```
CREATE VIEW Sales_Agg AS
SELECT
  p.category,
  c.city,
  SUM(s.quantity) AS total_qty,
  SUM(s.quantity * s.price) AS total_revenue
FROM Sale s
JOIN Product p ON s.product_id = p.product_id
JOIN Customer c ON s.customer_id = c.customer_id
GROUP BY p.category, c.city;
```

4. **Export to CSV**
   Export `Sales_Agg` (or any aggregated table) to `sales_agg.csv`.

Part B: Repeat with R

operations

First Example using Base R

Star Schema: 1 Fact table (Sales) + 4 Dimension tables (Date, Product, Customer, Store)

Then perform classic OLAP operations: Roll-up, Drill-down, Slice, Dice, Pivot

Uses ONLY base R (no external packages needed) so it runs anywhere.

set.seed(42)

[Sets the random number generator seed so that sample() and other random functions produce the **same results every time** you run the script. This is important for reproducibility in a lab]

```
## ------------------------------------------------------------------------

## 1. DIMENSION TABLES

## ------------------------------------------------------------------------

# --- Date Dimension ---

dim_date <- data.frame(

 date_id   = 1:24,

 date      = seq(as.Date("2024-01-01"), by = "month", length.out = 24),

 month     = rep(month.name, 2),

 quarter   = rep(paste0("Q", rep(1:4, each = 3)), 2),

 year      = rep(c(2024, 2025), each = 12)

)
```

[Creates 24 rows representing **monthly data from Jan 2024 to Dec 2025**.

Columns:

- date_id: primary key (1 to 24).
- date: actual Date values.
- month: month name (January–December, repeated twice).
- quarter: Q1–Q4, repeated twice (for 2 years).
- year: 2024 for first 12 rows, 2025 for next 12.

This is your **time dimension**, allowing analysis by month, quarter, or year.]

```
# --- Product Dimension ---

dim_product <- data.frame(

 product_id = 1:8,
```

```
  product_name = c("Laptop", "Smartphone", "Tablet", "Monitor",
                   "Desk Chair", "Office Desk", "Notebook", "Pen"),
  category = c("Electronics", "Electronics", "Electronics", "Electronics",
               "Furniture", "Furniture", "Stationery", "Stationery"),
  unit_price = c(55000, 22000, 18000, 9000, 4500, 8000, 60, 10)
)

# --- Customer Dimension ---
dim_customer <- data.frame(
  customer_id = 1:6,
  customer_name = c("Aarav", "Diya", "Kabir", "Meera", "Rohan", "Sneha"),
  segment = c("Corporate", "Individual", "Corporate", "Individual",
              "Individual", "Corporate")
)

# --- Store / Location Dimension ---
dim_store <- data.frame(
  store_id = 1:4,
  store_name = c("Chennai Store", "Bengaluru Store", "Mumbai Store", "Delhi Store"),
  region = c("South", "South", "West", "North")
)

## ------------------------------------------------------------------------

## 2. FACT TABLE (Sales) — generated to reference the dimension keys

## ------------------------------------------------------------------------
```

```
n <- 300  # number of transactions

fact_sales <- data.frame(

 sales_id    = 1:n,

 date_id     = sample(dim_date$date_id, n, replace = TRUE),

 product_id  = sample(dim_product$product_id, n, replace = TRUE),

 customer_id = sample(dim_customer$customer_id, n, replace = TRUE),

 store_id    = sample(dim_store$store_id, n, replace = TRUE),

 quantity    = sample(1:10, n, replace = TRUE)

)

# Bring in unit price to compute the measure "revenue"

fact_sales$unit_price <- dim_product$unit_price[match(fact_sales$product_id, dim_product$product_id)]

fact_sales$revenue    <- fact_sales$quantity * fact_sales$unit_price

cat("===== STAR SCHEMA CREATED =====\n")

cat("Fact table (fact_sales):", nrow(fact_sales), "rows\n")

cat("Dimensions: dim_date, dim_product, dim_customer, dim_store\n\n")

## ------------------------------------------------------------------------

## 3.BUILD A DENORMALIZED "OLAP CUBE" VIEW BY JOINING FACT + DIMENSIONS

## ------------------------------------------------------------------------



cube <- merge(fact_sales, dim_date,     by = "date_id")

cube <- merge(cube,       dim_product,  by = "product_id")
```

```
cube <- merge(cube,      dim_customer, by = "customer_id")

cube <- merge(cube,      dim_store,    by = "store_id")



cat("===== CUBE (joined fact + all dimensions) — first 5 rows =====\n")

print(head(cube[, c("date","product_name","category","customer_name",

                    "store_name","region","quantity","revenue")], 5))

cat("\n")
```

```
# Alternative Snowflake Schema

################################################################################

# BUILD A DATA WAREHOUSE IN R (SNOWFLAKE SCHEMA)

# Fact table: Sales

# Dimensions (normalized):

#   Date

#   Category -> Product

#   Segment  -> Customer

#   Region   -> Store

################################################################################

set.seed(42)

## ----------------------------------------------------------------------------

## 1. NORMALIZED DIMENSION TABLES

## ----------------------------------------------------------------------------
```

```
# --- Date Dimension (can stay as one table) ---
dim_date <- data.frame(
  date_id   = 1:24,
  date      = seq(as.Date("2024-01-01"), by = "month", length.out = 24),
  month     = rep(month.name, 2),
  quarter   = rep(paste0("Q", rep(1:4, each = 3)), 2),
  year      = rep(c(2024, 2025), each = 12)
)
# --- Category Dimension ---
dim_category <- data.frame(
  category_id = 1:3,
  category_name = c("Electronics", "Furniture", "Stationery")
)
# --- Product Dimension (references Category) ---
dim_product <- data.frame(
  product_id = 1:8,
  product_name = c("Laptop", "Smartphone", "Tablet", "Monitor",
                   "Desk Chair", "Office Desk", "Notebook", "Pen"),
  category_id = c(1, 1, 1, 1,   # Electronics
                  2, 2,         # Furniture
                  3, 3),        # Stationery
  unit_price = c(55000, 22000, 18000, 9000, 4500, 8000, 60, 10)
```

```
)

# --- Segment Dimension ---

dim_segment <- data.frame(

 segment_id = 1:2,

 segment_name = c("Corporate", "Individual")

)

# --- Customer Dimension (references Segment) ---

dim_customer <- data.frame(

 customer_id = 1:6,

 customer_name = c("Aarav", "Diya", "Kabir", "Meera", "Rohan", "Sneha"),

 segment_id = c(1, 2, 1, 2, 2, 1)  # Corporate=1, Individual=2

)

# --- Region Dimension ---

dim_region <- data.frame(

 region_id = 1:3,

 region_name = c("South", "West", "North")

)

# --- Store Dimension (references Region) ---

dim_store <- data.frame( store_id = 1:4,

 store_name = c("Chennai Store", "Bengaluru Store", "Mumbai Store", "Delhi Store"),

 region_id = c(1, 1, 2, 3)  # South=1, West=2, North=3

)
```

```
## ------------------------------------------------------------------------

## 2. FACT TABLE (same structure as before)

## ------------------------------------------------------------------------

n <- 300

fact_sales <- data.frame(

  sales_id    = 1:n,

  date_id     = sample(dim_date$date_id, n, replace = TRUE),

  product_id  = sample(dim_product$product_id, n, replace = TRUE),

  customer_id = sample(dim_customer$customer_id, n, replace = TRUE),

  store_id    = sample(dim_store$store_id, n, replace = TRUE),

  quantity    = sample(1:10, n, replace = TRUE)

)

# Bring in unit price to compute revenue

fact_sales$unit_price <- dim_product$unit_price[  match(fact_sales$product_id, dim_product$product_id)]

fact_sales$revenue    <- fact_sales$quantity * fact_sales$unit_price

cat("===== SNOWFLAKE SCHEMA CREATED =====\n")

cat("Fact table (fact_sales):", nrow(fact_sales), "rows\n")

cat("Dimensions:\n")

cat("  dim_date\n")

cat("  dim_category, dim_product\n")

cat("  dim_segment,  dim_customer\n")

cat("  dim_region,   dim_store\n\n")
```

```
## ------------------------------------------------------------------------

## 3. BUILD A DENORMALIZED "OLAP CUBE" VIEW (for analysis)

## ------------------------------------------------------------------------


# Join fact with all leaf dimensions and their sub-dimensions

cube <- merge(fact_sales, dim_date,     by = "date_id")

cube <- merge(cube,      dim_product,  by = "product_id")

cube <- merge(cube,      dim_category, by = "category_id")

cube <- merge(cube,      dim_customer, by = "customer_id")

cube <- merge(cube,      dim_segment,  by = "segment_id")

cube <- merge(cube,      dim_store,    by = "store_id")

cube <- merge(cube,      dim_region,   by = "region_id")


cat("===== CUBE (joined fact + all dimensions) — first 5 rows =====\n")

print(head(cube[, c("date","product_name","category_name","customer_name",

                    "segment_name","store_name","region_name","quantity","revenue")], 5))

cat("\n")
```

OPERATIONS

(a) Roll-up

Star:

```
rollup_year <- aggregate(revenue ~ year, data = cube, sum)
```

Snowflake:

Same syntax, just ensure `cube` includes `year` (from `dim_date`):

```
rollup_year <- aggregate(revenue ~ year, data = cube, sum)
```

No change in operation syntax.

(b) Drill-down

Star:

```
drilldown <- aggregate(revenue ~ category + product_name, data = cube, sum)
```

Snowflake:

Same, but now `category` comes from `dim_category`:

```
drilldown <- aggregate(revenue ~ category_name + product_name, data = cube, sum)
```

Only the **column name** changes (`category` → `category_name`), not the operation pattern.

(c) Slice

Star:

```
slice_south <- subset(cube, region == "South")
```

Snowflake:

```
slice_south <- subset(cube, region_name == "South")
```

Again, same `subset()` syntax, just different column name.

(d) Dice

Star:

```
dice <- subset(cube, region %in% c("South", "West") &
                category == "Electronics" &    year == 2025)
```

Snowflake:

```
dice <- subset(cube, region_name %in% c("South", "West") &
                category_name == "Electronics" & year == 2025)
```

Same logical structure, same functions.

(e) Pivot

Star:

```
pivot <- xtabs(revenue ~ category + region, data = cube)
```

Snowflake:

```
pivot <- xtabs(revenue ~ category_name + region_name, data = cube)
```

Same `xtabs()` syntax, just different column names.

```
## ------------------------------------------------------------------------
```

SAVE THE WAREHOUSE TABLES TO CSV (so they can be loaded elsewhere, e.g. imported into WEKA for the next ##part of the lab)

```
## ------------------------------------------------------------------------
```

Star Schema

```
out_dir <- "warehouse_output"

dir.create(out_dir, showWarnings = FALSE)

write.csv(dim_date,     file.path(out_dir, "dim_date.csv"),     row.names = FALSE)

write.csv(dim_product,  file.path(out_dir, "dim_product.csv"),  row.names = FALSE)

write.csv(dim_customer, file.path(out_dir, "dim_customer.csv"), row.names = FALSE)

write.csv(dim_store,    file.path(out_dir, "dim_store.csv"),    row.names = FALSE)

write.csv(fact_sales,   file.path(out_dir, "fact_sales.csv"),   row.names = FALSE)

write.csv(cube,         file.path(out_dir, "sales_cube_denormalized.csv"), row.names = FALSE)

cat("\nAll warehouse tables written to '", out_dir, "/' as CSV.\n", sep = "")
```

SNOWFLAKE Schema

```
out_dir <- "warehouse_output_snowflake"

dir.create(out_dir, showWarnings = FALSE)


write.csv(dim_date,     file.path(out_dir, "dim_date.csv"),     row.names = FALSE)

write.csv(dim_category,  file.path(out_dir, "dim_category.csv"),  row.names = FALSE)

write.csv(dim_product,   file.path(out_dir, "dim_product.csv"),   row.names = FALSE)

write.csv(dim_segment,   file.path(out_dir, "dim_segment.csv"),   row.names = FALSE)

write.csv(dim_customer,  file.path(out_dir, "dim_customer.csv"),  row.names = FALSE)

write.csv(dim_region,    file.path(out_dir, "dim_region.csv"),    row.names = FALSE)

write.csv(dim_store,     file.path(out_dir, "dim_store.csv"),     row.names = FALSE)

write.csv(fact_sales,    file.path(out_dir, "fact_sales.csv"),    row.names = FALSE)

write.csv(cube, file.path(out_dir, "sales_cube_denormalized_snowflake.csv"),

        row.names = FALSE)


cat("\nAll snowflake warehouse tables written to '", out_dir, "/' as CSV.\n", sep = "")
```

Second Example using SQL

```
# Install packages if you don't have them already
# install.packages("RSQLite")
# install.packages("DBI")
library(DBI)
library(RSQLite)
```

==========

1 & 2. Create Database, Tables, and Data

==========

Create an in-memory SQLite database

con <- dbConnect(RSQLite::SQLite(), ":memory:")

Define and execute table creation queries

```
dbExecute(con, "
 CREATE TABLE Product (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT,
    category TEXT
 );
")

dbExecute(con, "
 CREATE TABLE Customer (
    customer_id INTEGER PRIMARY KEY,
    name TEXT,
    city TEXT
 );
")
```

```
dbExecute(con, "
 CREATE TABLE Sale (
    sale_id INTEGER PRIMARY KEY,
    product_id INTEGER,
    customer_id INTEGER,
    date TEXT,
    quantity INTEGER,
    price REAL,
    FOREIGN KEY(product_id) REFERENCES Product(product_id),
    FOREIGN KEY(customer_id) REFERENCES Customer(customer_id)
 );
")
```

Populate Product table (5 sample rows)

```
products <- data.frame(
 product_id = 1:5,
 product_name = c("Laptop", "Smartphone", "Desk Chair", "Coffee Maker", "Headphones"),
 category = c("Electronics", "Electronics", "Furniture", "Appliances", "Electronics")
)
dbWriteTable(con, "Product", products, append = TRUE, row.names = FALSE)
```

Populate Customer table (5 sample rows)

```
customers <- data.frame(
 customer_id = 1:5,
 name = c("Alice Smith", "Bob Jones", "Charlie Brown", "Diana Prince", "Evan Wright"),
 city = c("New York", "Los Angeles", "New York", "Chicago", "Los Angeles")
)
dbWriteTable(con, "Customer", customers, append = TRUE, row.names = FALSE)


# Populate Sale table (10 sample rows)
sales <- data.frame(
 sale_id = 1:10,
 product_id = c(1, 2, 1, 3, 4, 5, 2, 4, 3, 1),
```

```
  customer_id = c(1, 2, 3, 4, 5, 1, 3, 2, 5, 4),
  date = c("2026-01-01", "2026-01-02", "2026-01-03", "2026-01-04", "2026-01-05",
           "2026-01-06", "2026-01-07", "2026-01-08", "2026-01-09", "2026-01-10"),
  quantity = c(1, 2, 1, 4, 1, 3, 1, 2, 2, 1),
  price = c(1200.00, 800.00, 1200.00, 150.00, 80.00, 150.00, 800.00, 80.00, 150.00, 1200.00)
)
dbWriteTable(con, "Sale", sales, append = TRUE, row.names = FALSE)
```

==========

3. Create the Star-Schema View

==========

```
dbExecute(con, "
  CREATE VIEW Sales_Agg AS
  SELECT
     p.category,
     c.city,
     SUM(s.quantity) AS total_qty,
     SUM(s.quantity * s.price) AS total_revenue
  FROM Sale s
  JOIN Product p ON s.product_id = p.product_id
  JOIN Customer c ON s.customer_id = c.customer_id
  GROUP BY p.category, c.city;
")
```

==========

4. Fetch data from View and Export to CSV

==========

Query the view back into R

```
sales_agg_df <- dbGetQuery(con, "SELECT * FROM Sales_Agg;")
```

Print to console to inspect the result

```
print(sales_agg_df)
```

Export to a CSV file in your working directory

```
write.csv(sales_agg_df, "sales_agg.csv", row.names = FALSE)
```

Always disconnect from the database when done

```
dbDisconnect(con)
```

Solution Notes

- Data warehouse: central repository for integrated data from multiple sources, optimized for querying and analysis.
- In WEKA, you don't implement the warehouse itself; you load its exported data to explore and apply algorithms.
- Key concepts: schema, ETL, star schema, OLAP-style aggregation.

CLASS WORK for evaluation

Experiment 1: Build Data Warehouse and Explore WEKA

Real-Life Problem: Retail Sales Analytics for a Chain Store

Context:

A retail chain in Tamil Nadu sells electronics and accessories across multiple cities (Madurai, Chennai, Coimbatore). They want to analyze sales by product category and city to understand which combinations are most profitable.

Dataset Structure:

- `product(product_id, product_name, category)`
- `customer(customer_id, name, city)`
- `sale(sale_id, product_id, customer_id, date, quantity, price)`

Derived Aggregated Table (for data warehouse):

- `sales_agg(category, city, total_qty, total_revenue)`

How it maps:

- Students design a star-schema data warehouse.
- They create an aggregated view in SQL.
- Export to CSV/ARFF and explore in WEKA (attribute distributions, basic plots).

Possible practice experiments:

1. Designing a Snow-flake schema for the same.

2. Create proper views.

3. Export data in CSVs.

Explore Data in WEKA

1. Open WEKA Gui.
2. Click **Open file**, select `sales_agg.csv`.
3. In the **Preprocess** panel:
    - Observe the list of attributes.
    - Click on each attribute to see value distribution.
4. Try **Visualize** button to see scatter plots.

Expected Output

- A small data warehouse (tables + view).
- CSV file exported.
- WEKA showing attributes, distributions, and basic plots.